

Artificial intelligence

Adversarial Arena

TEAM MEMBERS:

- 1- عبدالرحمن احمد محمد عبدالنبي السيد
- 2- عبدالرحمن خالد حسن ابووالي جرامون
- 3- عبدالله عماد الدساوي محمد
- 4- عبدالرحمن حسام الدين شعبان
- 5- عبدالرحمن معوض محمد عبدالواحد

Adversarial-Arena – Project Documentation

Project Overview

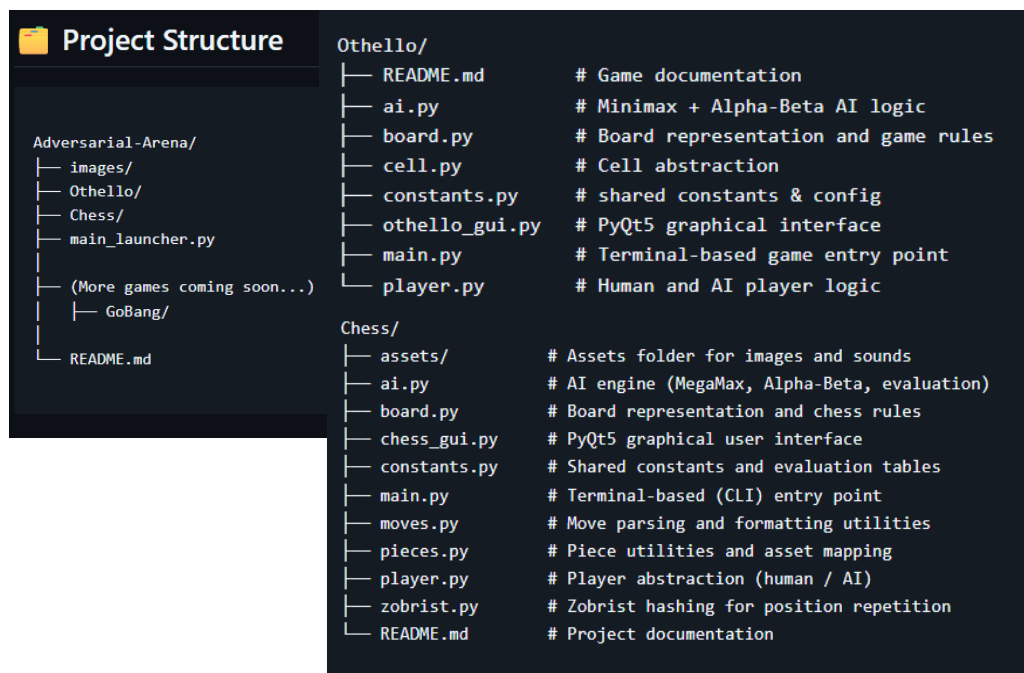
Adversarial-Arena is an academic and experimentation-oriented project that focuses on the design and implementation of adversarial board games using classical Artificial Intelligence search techniques.

The project serves as a unified arena where multiple turn-based games are implemented under a shared architectural philosophy while remaining logically independent.

The primary motivation behind this project is to bridge the gap between theoretical AI concepts and their practical application in real game environments. Rather than relying on machine learning or neural networks, the project emphasizes deterministic, explainable, and classical AI approaches such as Minimax, NegaMax, and Alpha-Beta pruning. The project is designed to be extensible, allowing new games and strategies to be added incrementally without affecting existing modules.

Project Structure

The repository is organized in a modular structure where each game exists as an independent package. This design ensures clean separation of responsibilities and prevents tight coupling between different game implementations.



At the root level, a unified launcher is provided to allow users to select and run any available game through a single graphical interface. Shared architectural concepts such as turn-based execution, AI-driven decision making, and GUI interaction are consistently applied across all games

Methodology & Artificial Intelligence Design

The development of the Adversarial-Arena project followed a structured, incremental, and modular methodology

The overall development process was designed to be consistent across all games while allowing each game to maintain its own specific logic, rules, and AI strategy.

At a high level, the project was built by first analyzing the rules and mechanics of each game, then designing an internal board representation suitable for efficient state evaluation. After that, an appropriate adversarial AI approach was selected for each game based on its strategic characteristics. Finally, a graphical user interface was implemented to provide an interactive and user-friendly experience.

This unified methodology ensures architectural consistency while allowing clear separation between game logic, AI logic, and user interface components.

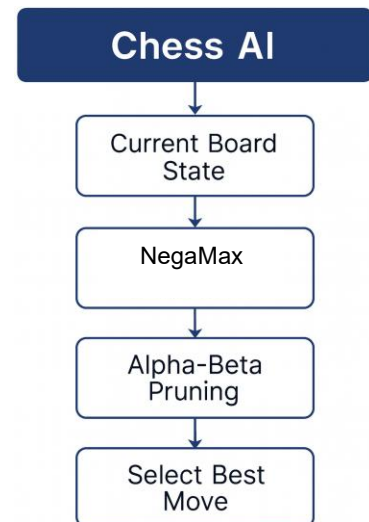
The following sections describe the methodology and AI design for each game separately, allowing clear documentation and the inclusion of game-specific diagrams and screenshots.

Chess – Methodology & AI Design

The development of the Chess module began with a detailed modeling of standard chess rules, including legal move generation, special moves, and game termination conditions. A structured board representation was designed to accurately reflect the state of the game and allow efficient evaluation during search.

For decision making, the Chess AI is based on the **NegaMax** adversarial search algorithm combined with **Alpha-Beta pruning**. This approach was chosen to provide a unified and deterministic perspective of the game state, ensuring that identical board positions and search depths always result in the same move. The evaluation strategy considers material balance, positional advantages through piece-square tables, and overall strategic stability rather than short-term tactics alone.

This design allows the Chess AI to remain predictable, explainable, and suitable for academic analysis, while maintaining acceptable performance within practical search depths.

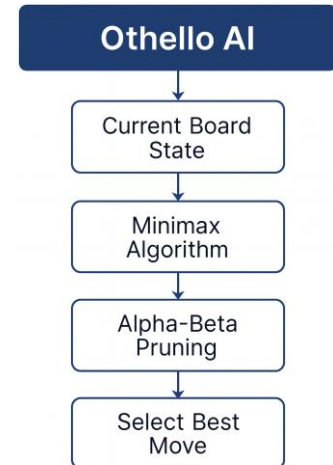


Othello / Reversi – Methodology & AI Design

The Othello module was developed by first analyzing the core mechanics of the game, including valid move detection, piece flipping, turn skipping, and game termination. A board representation was designed to support efficient scanning in all directions to detect legal moves and apply flips correctly.

The AI player in Othello uses the **Minimax** algorithm with **Alpha-Beta pruning** to explore future game states and evaluate strategic outcomes. The evaluation function focuses on board control, positional weighting using heatmaps, and long-term stability rather than immediate piece gain. This approach allows the AI to favor strong positional moves such as corners and stable edges.

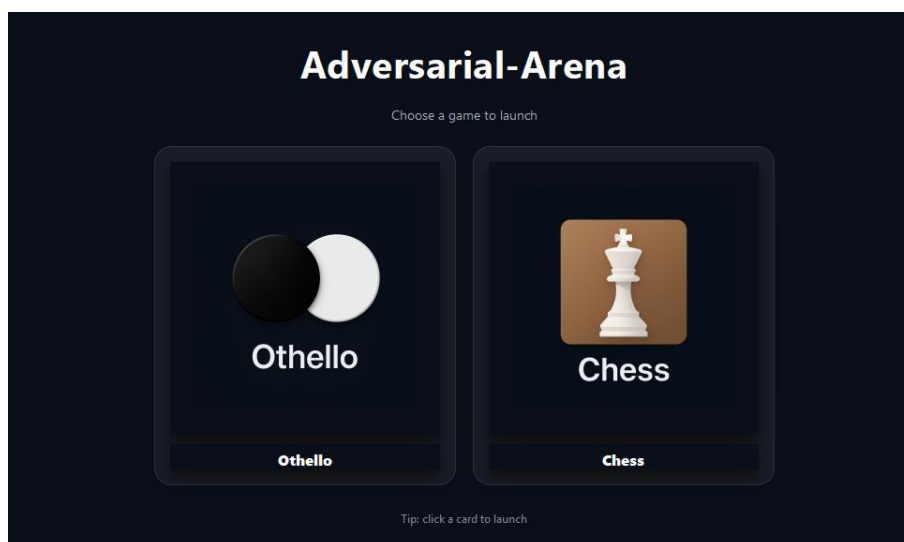
By separating the AI logic from both the game rules and the user interface, the Othello module maintains clarity, reusability, and ease of testing across different gameplay modes.



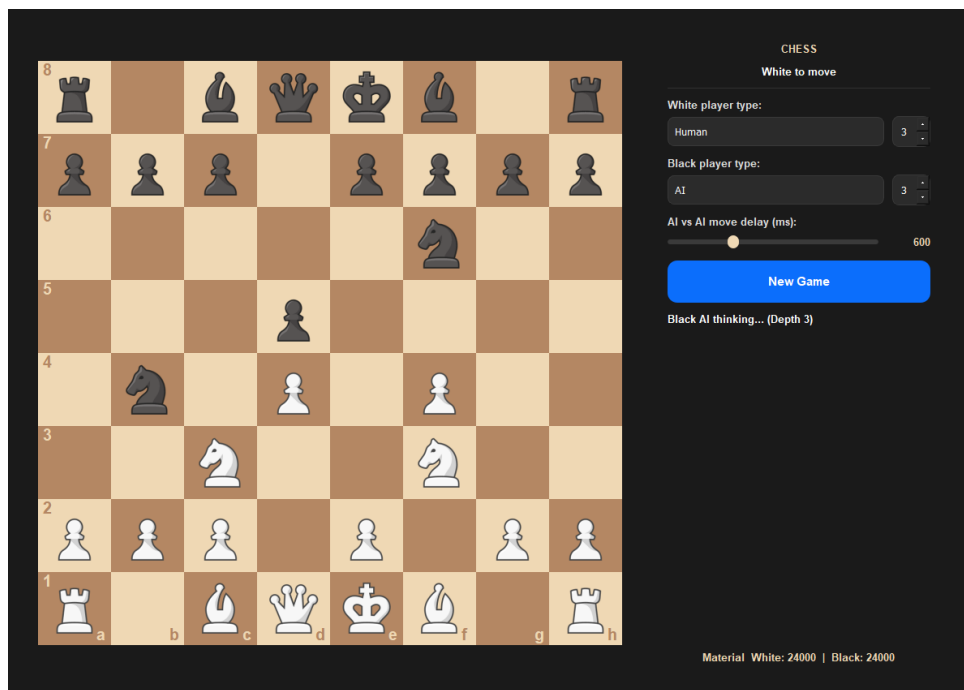
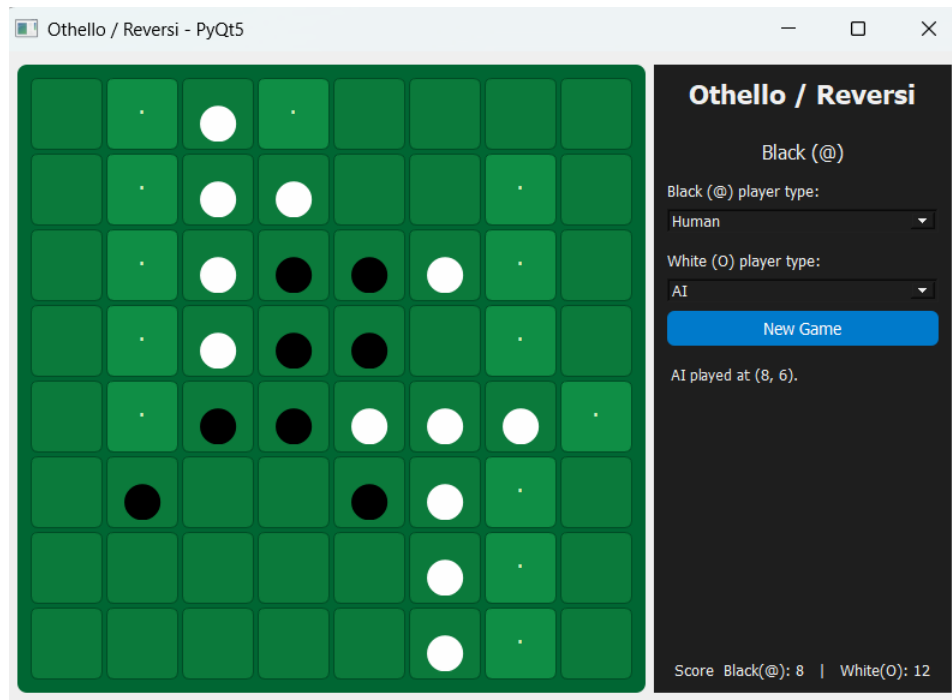
Graphical Interface, Execution & Usage

All graphical interfaces in the Adversarial-Arena project are implemented using **PyQt5**, where each game provides its own dedicated graphical interface while following a unified visual and interaction style across the entire project. The graphical layer is fully separated from both the game logic and AI components, allowing each part to be developed, tested, and maintained independently.

The project includes a **unified launcher** that serves as the main entry point to the system. Through this launcher, users can select and run any available game (such as Chess or Othello) from a single window, without the need to manually navigate directories. This approach provides a centralized and user-friendly experience while still preserving the independence of each game module.



In addition to the unified launcher, each game can also be executed **independently** using its own graphical interface. This allows focused interaction with a specific game and supports direct testing and debugging without loading the entire arena environment. Each game also offers a **CLI mode** that runs on the same core logic, providing a lightweight alternative for testing and analysis.



Testing and Validation

Testing was conducted through extensive manual gameplay, including Human vs AI and AI vs AI matches. Special attention was given to validating deterministic behavior, rule enforcement, and game termination conditions. Repeated executions using identical initial conditions were used to confirm the consistency of AI decision-making.

Challenges and Solutions

During the development of the Adversarial-Arena project, several technical and organizational challenges were encountered.

One of the main technical challenges was the **AI computation time**, especially in scenarios involving deeper search levels and AI vs AI gameplay. Adversarial search algorithms such as Minimax and NegaMax require exploring a large number of game states, which can lead to noticeable delays during decision-making. This challenge was addressed by applying **Alpha-Beta pruning**, optimizing move ordering, and carefully selecting reasonable default search depths to balance performance and decision quality.

Another significant challenge involved **project organization and team collaboration**. Since different team members were responsible for different components (such as individual games, AI logic, and graphical interfaces), it was necessary to design a clear and modular file structure that allowed each member to work independently without interfering with others' progress. This was resolved by treating each game as a self-contained package with a clear separation between game logic, AI logic, and user interface, enabling smooth integration of all components into a unified project.

Additionally, combining independently developed modules into a single working system required careful handling of **file structure and module dependencies**. Import conflicts and integration issues initially arose when merging the different parts of the project. These issues were overcome by adopting a consistent package-based structure and enforcing clear interface boundaries between modules.

Finally, ensuring that the **graphical interface remained responsive** during AI computations posed another challenge. This was mitigated through controlled execution flow and optimization of the AI search process, ensuring that gameplay remained stable and user-friendly even during complex AI decision-making scenarios.

Conclusion

Adversarial-Arena successfully demonstrates the application of classical AI search techniques within interactive board game environments. The project emphasizes clean architecture, deterministic AI behavior, and extensibility, making it suitable for both educational purposes and portfolio presentation.